CXL on Linux: Boot To Bash

This documentation is intended as a companion to systems engineers (Platform, BIOS, EFI, OS, etc) trying to enable CXL devices on Linux.

CXL device configuration requires platform (Hardware, BIOS, EFI), OS (early boot, core kernel, driver), and user policy decisions that all impact each other. This doc breaks up these configurations into five main areas:

Overview

The type of CXL device (Memory, Accelerator, etc) dictates many configuration steps. This section covers some basic background on device types and on-device resources used by the platform and OS which impact configuration.

Devices Types

Memory Expanders

todo

Persistent Memory

todo

Volatile Memory

todo

Logical Devices

todo

Single-Logical Device (SLD)

todo

Multi-Logical Device (MLD)

todo

Multi-Headed Devices

todo

MHSLD

todo

MHMLD

todo

Dynamic Capacity Devices

todo

Memory Extents

todo

Shared Memory

todo

UEFI Data

Coherent Device Attribute Table (CDAT)

todo

Overview

Hardware, BIOS, and EFI are all responsible for configuring static information about devices (or potential future devices) such that Linux can build the appropriate logical representations of these devices. Much of what this section is concerned with is ACPI Table production and static memory map configuration.

At a high level, this is what occurs during this phase of configuration.

- The bootloader starts the EFI / BIOS
- BIOS / EFI do early device probe to determine static configuration
- BIOS / EFI creates ACPI Tables that describe static config for the OS
- Calls `start_kernel` and begins the Linux Early Boot process.

EFI Configuration Process

The BIOS (Basic Input Output System) / EFI (Extensible Firmware Interface) is the first thing that comes up when the system boots. More explicitly, it is what the bootloader hands off to initiate the rest of startup. `start_kernel` is called in this step, which starts Linux.

Together, EFI and BIOS program the initial ACPI tables. Notably, they generate the CEDT (CXL Early Discovery Table) and SRAT (System Resource Affinity Table). More detail on these tables can be found under Platform Configuration -> ACPI Table Reference.

UEFI Settings

The `uefisettings` command can be used to retrieve / set EFI settings. Once changes are made, rebooting the machine incorporates the changes made into the next boot.

One notable configuration here is the EFI Memory SP (Special Purpose) bit. When this is enabled, it means that the driver is responsible for managing the memory region. Otherwise, the memory is treated as “normal memory”, and is exposed to the page allocator.

uefisettings examples

`uefisettings identify`

```
uefisettings identify

bios_vendor: xxx
bios_version: xxx
bios_release: xxx
bios_date: xxx
product_name: xxx
product_family: xxx
product_version: xxx
```

`uefisettings get "CXL Physical Addressing"`

```
selector: xxx
...
question: Question {
    name: "CXL Memory Attribute",
    answer: "Enabled",
    ...
}
```

Physical Memory Map

Physical Address Region Alignment

As of Linux v6.14, the hotplug memory system requires memory regions to be uniform in size and alignment. While the CXL specification allows for memory regions as small as 256MB, the supported memory block size and alignment is architecture-defined. Blocks may be as small as 128MB and increase uniformly as a power of two.

On ARM, the default block size and alignment is either 128MB or 256MB.

On x86, the default block size is 256MB, and increases to 2GB as the capacity of the system increases up to 64GB.

For best support, platform vendors should place CXL memory at a 2GB aligned base address, and regions should be 2GB aligned.

Memory Holes

Holes in the memory map are tricky. Consider a 4GB device set at base 0x100000000, set up in the following memory map

```
---------------------
|    0x100000000    |
|                   |
|        CXL        |
|                   |
|    0x1BFFFFFFF    |
---------------------
|    0x1C0000000    |
|    MEMORY HOLE    |
|    0x1FFFFFFFF    |
---------------------
|    0x200000000    |
|                   |
|     CXL CONT.     |
|                   |
|    0x23FFFFFFF    |
---------------------
```

There are two issues to consider: decoder programming and memory block alignment.

If your architecture requires 2GB uniform size and aligned memory blocks, the only capacity Linux is capable of mapping (as of v6.14) would be the capacity from *0x100000000-0x180000000*. The remaining capacity will be stranded, as they are not of 2GB aligned length.

Assuming your architecture and memory configuration allows 1GB memory blocks, this memory map is supported and this should be presented as multiple CFMWS in the CEDT that describe each side of the memory hole separately - along with matching decoders.

Each endpoint intended to be mapped into this layout should plan to use multiple decoders as well.

Decoder Programming

If BIOS/EFI intends to program the decoders to be statically configured, there are a few things to consider to avoid major pitfalls that will prevent Linux compatibility. These recommendations are not required “per the specification”, but Linux makes no guarantees of support otherwise.

Translation Point

Per the specification, the only decoders which *translate* Host Physical Address to Device Physical Address are the **Endpoint Decoders**. All other decoders in the fabric are intended to route accesses without translating the addresses.

Due to some ambiguity in how Architecture, ACPI, PCI, and CXL specifications “hand off” responsibility between domains, some early adopting platforms attempted to do translation at the originating memory controller or host bridge. This configuration requires a platform specific extension to the driver and is not officially endorsed - despite being supported.

It is highly recommended *not* to do this; otherwise, you are on your own to provide driver support for your platform.

Interleave and Configuraiton Flexibility

If providing cross-host-bridge interleave, a CFMWS entry in the CEDT must be presented with target host-bridges for the interleaved device sets (there may be multiple behind each host bridge).

If providing intra-host-bridge interleaving, only 1 CFMWS entry in the CEDT is required for that host bridge - if it covers the entire capacity of the devices behind the host bridge.

If intending to provide users flexibility in programming decoders beyond the root, you may want to provide multiple CFMWS entries in the CEDT intended for different purposes. For example, you may want to consider adding

1. A CFMWS entry to cover all interleavable host bridges.
2. A CFMWS entry to cover all devices on a single host bridge.
3. A CFMWS entry to cover each device.

A platform may choose to add all of these, or change the mode based on a BIOS setting. For each CFMWS entry, Linux expects descriptions of the described memory regions in the SRAT to determine the number of NUMA nodes it should reserve during early boot / init.

Memory Holes

If your platform includes memory holes intersparsed between your CXL memory, it is recommended to utilize multiple decoders to cover these regions of memory, rather than try to program the decoders to accept the entire range and expect Linux to manage the overlap.

Linux makes no guarantee of support for strange memory hole situations.

Multi-Media Devices

Devices that have either:

1. A multi-purpose media (i.e. persistent mem used as volatile), or
2. Multiple forms of memory

Require special restriction bits (specifically Volatile vs Persistent bits) set in the CFMWS entries in the CEDT.

A CFMWS may be set to allow either persistent or volatile, but for best flexibility platforms may wish to define multiple CFMWS entries to allow flexible configuration by a user at runtime.

ACPI Tables

ACPI is the “Advanced Configuration and Power Interface”, which is a standard that defines how platforms and OS manage pwoer and configure computer hardware. For the purpose of this theory of operation, when referring to “ACPI” we will usually refer to “ACPI Tables” - which are the way a platform (BIOS/EFI) communicates static configuration information to the operation system.

The Following ACPI tables contain *static* configuration and performance data about CXL devices.

CEDT - CXL Early Discovery Table

The CXL Early Discovery Table is generated by BIOS to describe the CXL memory regions configured at boot by the BIOS.

CHBS

The CXL Host Bridge Structure describes CXL host bridges. Other than describing device register information, it reports the specific host bridge UID for this host bridge. These host bridge ID's will be referenced in other tables.

Example

```
          Subtable Type : 00 [CXL Host Bridge Structure]
               Reserved : 00
                 Length : 0020
Associated host bridge : 00000007    <- Host bridge _UID
 Specification version : 00000001
               Reserved : 00000000
          Register base : 0000010370400000
       Register length : 0000000000010000
```

CFMWS

The CXL Fixed Memory Window structure describes a memory region associated with one or more CXL host bridges (as described by the CHBS). It additionally describes any inter-host-bridge interleave configuration that may have been programmed by BIOS.

Example

```
            Subtable Type : 01 [CXL Fixed Memory Window Structure]
                 Reserved : 00
                   Length : 002C
                 Reserved : 00000000
     Window base address : 000000C050000000   <- Memory Region
             Window size : 0000003CA0000000
Interleave Members (2^n) : 01                 <- Interleave configuration
   Interleave Arithmetic : 00
                 Reserved : 0000
             Granularity : 00000000
            Restrictions : 0006
                   QtgId : 0001
            First Target : 00000007           <- Host Bridge _UID
             Next Target : 00000006           <- Host Bridge _UID
```

The restriction field dictates what this SPA range may be used for (memory type, voltile vs persistent, etc). One or more bits may be set.

```
Bit[0]: CXL Type 2 Memory
Bit[1]: CXL Type 3 Memory
Bit[2]: Volatile Memory
Bit[3]: Persistent Memory
Bit[4]: Fixed Config (HPA cannot be re-used)
```

INTRA-host-bridge interleave (multiple devices on one host bridge) is NOT reported in this structure, and is solely defined via CXL device decoder programming (host bridge and endpoint decoders).

SRAT - Static Resource Affinity Table

The System/Static Resource Affinity Table describes resource (CPU, Memory) affinity to "Proximity Domains". This table is technically optional, but for performance information (see "HMAT") to be enumerated by linux it must be present.

There is a careful dance between the CEDT and SRAT tables and how NUMA nodes are created. If things don't look quite the way you expect - check the SRAT Memory Affinity entries and CEDT CFMWS to determine what your platform actually supports in terms of flexible topologies.

The SRAT may statically assign portions of a CFMWS SPA range to a specific proximity domains. See linux numa creation for more information about how this presents in the NUMA topology.

Proximity Domain

A proximity domain is ROUGHLY equivalent to "NUMA Node" - though a 1-to-1 mapping is not guaranteed. There are scenarios where "Proximity Domain 4" may map to "NUMA Node 3", for example. (See "NUMA Node Creation")

Memory Affinity

Generally speaking, if a host does any amount of CXL fabric (decoder) programming in BIOS - an SRAT entry for that memory needs to be present.

Example

```
         Subtable Type : 01 [Memory Affinity]
                Length : 28
     Proximity Domain : 00000001          <- NUMA Node 1
            Reserved1 : 0000
          Base Address : 000000C050000000  <- Physical Memory Region
        Address Length : 0000003CA0000000
            Reserved2 : 00000000
Flags (decoded below) : 0000000B
              Enabled : 1
        Hot Pluggable : 1
         Non-Volatile : 0
```

Generic Initiator / Port

todo

HMAT - Heterogeneous Memory Attribute Table

The Heterogeneous Memory Attributes Table contains information such as cache attributes and bandwidth and latency details for memory proximity domains. For the purpose of this document, we will only discuss the SSLIB entry.

SLLBI

The System Locality Latency and Bandwidth Information records latency and bandwidth information for proximity domains.

This table is used by Linux to configure interleave weights and memory tiers.

Example (Heavily truncated for brevity)

```
              Structure Type : 0001 [SLLBI]
                   Data Type : 00         <- Latency
Target Proximity Domain List : 00000000
Target Proximity Domain List : 00000001
                       Entry : 0080       <- DRAM LTC
                       Entry : 0100       <- CXL LTC

              Structure Type : 0001 [SLLBI]
                   Data Type : 03         <- Bandwidth
Target Proximity Domain List : 00000000
Target Proximity Domain List : 00000001
                       Entry : 1200       <- DRAM BW
                       Entry : 0200       <- CXL BW
```

SLIT - System Locality Information Table

The system locality information table provides “abstract distances” between accessor and memory nodes. Node without initiators (cpus) are infinitely (FF) distance away from all other nodes.

The abstract distance described in this table does not describe any real latency of bandwidth information.

Example

```
   Signature : "SLIT"    [System Locality Information Table]
  Localities : 0000000000000004
Locality   0 : 10 20 20 30
Locality   1 : 20 10 30 20
Locality   2 : FF FF 0A FF
Locality   3 : FF FF FF 0A
```

DSDT - Differentiated system Description Table

This table describes what peripherals a machine has.

This table's UIDs for CXL devices - specifically host bridges, must be consistent with the contents of the CEDT, otherwise the CXL driver will fail to probe correctly.

todo: example table entry for CXL
The SRAT table may also contain generic port/initiator content that is intended to describe the generic port, but not information about the rest of the path to the endpoint.

Linux uses these tables to configure kernel resources for statically configured (by BIOS/EFI) CXL devices, such as:

- NUMA nodes
- Memory Tiers
- NUMA Abstract Distances
- SystemRAM Memory Regions
- Weighted Interleave Node Weights

ACPI Debugging

The `acpidump` command can be used at runtime to dump the contents of the tables that have been created. Running the command by itself displays the hex contents of the ACPI to stdout. Not very helpful :_(

Running `acpidump -b` will dump out the tables in individual .dat files, which `iasl -d` (disassemble) can use to convert into .dsl files, showing human-readable tables.

acpidump

On a machine with CXL: `acpidump -b && iasl -d cedt.dat`

```
/*
 * Intel ACPI Component Architecture
 * AML/ASL+ Disassembler version 20210604 (64-bit version)
 * Copyright (c) 2000 - 2021 Intel Corporation
 *
 * Disassembly of cedt.dat, Fri Apr 11 07:47:31 2025
 *
 * ACPI Data Table [CEDT]
 *
 * Format: [HexOffset DecimalOffset ByteLength]  FieldName : FieldValue
 */

[000h 0000   4]                    Signature : "CEDT"    [CXL Early Discovery Table]
...
```

Example Platform Configurations

One Device per Host Bridge

todo

Multiple Devices per Host Bridge

todo

Cross-Host-Bridge Interleave

todo

Multi-Media Devices

persistent and volatile

Overview

- Linux Boot Parameters
  - nosoftreserve
- Early Boot
  - Memory Map Creation
    - EFI Memory Map Is Consulted / Created
    - CXL Memory is set aside to be handled by the CXL driver
    - IO Resources are created for each piece
  - NUMA Node Creation
    - ACPI CEDT and SRAT table are used to create Nodes from Proximity domains (PXM)
  - Memory Tier Creation
    - A default tier is created with all nodes.
  - Contiguous Memory Allocation
    - Any requested CMA is allocated from Online nodes
  - Init Finishes, Drivers start probing
- ACPI and PCI Drivers
  - Detect CXL device, marking it for probe by CXL driver
- CXL Driver Operation
  - Base object creation (root, port, memdev)
    - CEDT CFMWS IO Resource creation
  - Decoder creation (root, switch, endpoint)
  - Logical device creation (region, endpoint)
  - Auto-regions surfaced as DAX region
- DAX Driver Operation
  - DAX kmem driver surfaces dax device

        - DAX kmem IO resource creation
    - DAX kmem surfaces memory region to Memory Hotplug to add to page allocator as “driver managed memory”
- Memory Hotplug
    - mhp component surfaces a memory region as multiple memory blocks to the page allocator
    - blocks are onlined into the requested zone (NORMAL or MOVABLE)
    - Memory is marked “Driver Managed” to avoid kexec from using it as region for kernel updates

Linux Init (Early Boot)

Linux configuration is split into two major steps: Early-Boot and everything else.

During early boot, Linux sets up immutible resources (such as numa nodes), while later operations include things like driver probe and memory hotplug. Linux may read EFI and ACPI information throughout this process to configure logical representations of the devices.

During Linux Early Boot stage (functions in the kernel that have the __init decorator), the system takes the resources created by EFI/BIOS (ACPI tables) and turns them into resources that the kernel can consume.

NUMA Nodes

Linux takes the PXM (proximity domain) table in the SRAT to create NUMA nodes in `acpi_numa_init`. Typically, there is a 1:1 correspondance between the proximity domains and NUMA nodes (exception: see `fake_pxm`. If there are CXL ranges in the CFMWS but not in SRAT, then a fake proximity domain is created). As nodes come online, memory tiers are created by grouping nodes with similar characteristics together.

NUMA nodes are *NOT* hot-pluggable. All *POSSIBLE* NUMA nodes are identified at *__init* time, more specifically during *mm_init*. What this means is that the CEDT and SRAT must contain sufficient *proximity domain* information for linux to identify how many NUMA nodes are required (and what memory regions should be associated with them).

SRAT is the only ACPI defined way of getting Proximity nodes. Linux chooses to, at most, map those 1:1 with NUMA nodes.

CEDT adds a description of SPA ranges which Linux may wish to map to one or more NUMA nodes

The relevant code exists in: linux/drivers/acpi/numa/srat.c

Basically, the heuristic is as follows

1. Add one NUMA node per Proximity Domain described in SRAT (one or more of: memory, cpu, generic initiator).
2. If SRAT describes some portion of a CFMWS SPA range - do not create a nodes for that CFMWS
3. If SRAT does not describe memory in a CFMWS SPA range - a node is created for that CFMWS

In the future, Linux may reject CFMWS not described by SRAT due to the ambiguity of proximity domain association.

Generally speaking, you will see one NUMA node per Host bridge, unless inter-host-bridge interleave is in use. In environments with complex QoS concerns, or designed for runtime configuration flexibility, it may be reasonable to create more than one NUMA node per Host bridge.

EFI_MEMORY_SP

While the kernel parses the EFI memory map in `do_add_efi_memmap` (x86), it also checks for the EFI MEM SP bit that was set during the EFI/BIOS stage. If the bit was set, then that memory range is set aside as `SOFT_RESERVED` i.e. defer management to the (CXL) driver. Otherwise, it will be onlined as `RAM` or `RESERVED`, depending on whether the EFI MEM WB (writeback) bit was set.

Memory Tiers Creation

todo

Contiguous Memory Allocation

todo: explain limitations

CXL Driver Operation

Base Objects

Root, Port, Memdev

Decoders

Root, Switch, Endpoint

Auto Decoders vs Decoder Programming

todo

Memory Regions

todo

Structure and Relationships

todo

Interleave Validation

todo

Surfacing Memory as DAX

todo

DAX Driver Operation

DAX Device

todo

kmem conversion

todo - memory hotplug interaction

Memory Hotplug

todo

DAX Devices

todo

The Page Allocator

todo: overall behavior

Memory Zones

todo

NUMA Nodes

todo

Memory Policy (mempolicy)

todo

CGroups and CPUSets

todo

Reclaim

todo: overview

Demotion

todo

ZSwap

todo

CXL-preferred ZSwap

todo

Memory Tiering

todo

Memory Tiers

todo

Transparent Page Placement

todo

Data Access MONitor

to be updated

References

- Self-tuned Memory Tiering RFC prototype and its evaluation
- SK Hynix HMSDK Capacity Expansion
- kernel documentation

- project website

Huge Pages

todo

Contiguous Memory Allocator

todo

HugeTLB

todo

2MB Huge Pages

todo

1GB Huge Pages

todo

Memory Expansion

todo

As Page Cache

todo

As DAX Device

todo

Dynamic Capacity

todo

For Virtual Machines

todo

For Workload Orchestration

todo

For Shared Memory

todo

Virtual Machines

todo

NUMA Passthrough

todo

Flexible Shapes

todo

Datacenter Efficiency

todo

Shared Memory

todo

Coherence

todo

Fabric Attached Memory FileSystem (FAMFS)

todo

Contributors

Authors

- Gregory Price
- Joshua Hahn

Editors

todo

Thanks

todo